UNIVERSITÀ DI TORINO

# Python for Data Science

## From Programming Basics to Machine Learning

Laboratorio di Python - 2025/2026

**Giulia Coucourde**

```python
def print_title():
    print("Python Course")
```

**Giulia Coucourde**
*PhD Student* - Dipartimento di informatica
**Research topic:** Integration of Structured Knowledge and Generative AI
**e-mail:** giulia.coucourde@unito.it

# Table of Contents

1 Introduction


- Introduction
- Practice Basics
- Python Basics

# Progettare in python



Basi per creare e gestire un progetto Python:

- Package manager (Windows e macOS): winget, homebrew
- Struttura di un progetto Python
- Ambienti virtuali: venv, miniconda, uv
- Version control con Git e Github

# Cos'è un Package Manager?



Tool per automatizzare la gestione e il controllo di installazione, aggiornamento, configurazione e rimozione di pacchetti software con le realtive dipendenze. *Esempio:* il nostro programma è una torta che necessita di alcuni ingredienti (dipendenze) per essere cucinata. Il packet manager gestisce tutti gli ingredienti al posto nostro.

- winget e homebrew gestiscono programmi nel sistema operativo.
- pip è il package installer per Python

# Terminologia per Python



- **Script:** Singolo file Python eseguibile tipicamente scritto per automatizzare qualcosa
- **Modulo:** File che contiene definizioni e istruzioni; python viene fornito con una libreria di moduli *standard*
- **Package:** Directory che contiene moduli Python e un file speciale chiamato `__init.py__`
- **Libreria:** Collezione di moduli

# Struttura di un progetto

## 1 Introduction

Due modi per strutturare un progetto python:

```
project_name/
├── bin/                    # Optional
│   └── project_name
├── docs/                   # Optional
│   ├── index.html
│   └── source.txt
├── src/
│   └── project_name/
│       ├── __init__.py
│       ├── __main__.py
│       ├── module1.py
│       └── module2.py
├── tests/
│   ├── __init__.py
│   ├── test_module1.py
│   └── test_module2.py
├── LICENSE
├── README.md
└── pyproject.toml
```

Figure: Src layout

```
project_name/
├── bin/                    # Optional
│   └── project_name
├── docs/                   # Optional
│   ├── index.html
│   └── source.txt
├── project_name/
│   ├── __init__.py
│   ├── __main__.py
│   ├── module1.py
│   └── module2.py
├── tests/
│   ├── __init__.py
│   ├── test_module1.py
│   └── test_module2.py
├── LICENSE
├── pyproject.toml
└── README.md
```

Figure: Flat layout

# Ambienti virtuali

1 Introduction

***Cosa sono?*** Directory isolate che contengono delle specifiche installazioni di Python con le relative dipendenze. Servono per gestire separatamente le dipendenze di progetti diversi in modo da evitare conflitti di versione.

**Applicazione A**

Python 3.11
Matplotlib 3.10.8
Pandas
Numpy

⚠️

**Applicazione B**

Python 3.14
Matplotlib 3.5.1
Pandas
Numpy

# Strumenti per la gestione degli ambienti

1 Introduction

**Virtualenv**

venv è parte delle Standard Library:
- Gestisce *solo* gli ambienti
- Non installa diverse versioni di Python (può usare solo un interprete già presente)
- Usa **pip** come package manager

**conda**

- Version Management
- Cross-language, cross-platform
- Usa sia conda che pip come packet manager
- Pesante, va bene per progetti grandi che usano più linguaggi

**uv**

- Version Management
- Sia package che project manager
- Installazione veloce e mantenimento delle dipendenze
- Project Initialization: ambienti, repository, README etc...

# Version Control: git

## 1 Introduction

Git è un sistema di versioning che permette di tracciare e gestire i cambiamenti nel codice:

> *Git considera i propri dati più come un flusso (sequenza) di istantanee (**snapshot**) di un mini filesystem. Con Git, ogni volta che registri (**commit**), o salvi lo stato del tuo progetto, fondamentalmente lui fa un'immagine di tutti i file in quel momento, salvando un riferimento allo **snapshot**.*

# Github



Estende le funzionalità di git per un completo development lifecycle:

- git: gestisci i tuoi cambiamenti in locale
- Github: sincronizzazione online; permette di condividere codice, gestire progetti a livello di team e migliora la coordinazione.

# Repository

## 1 Introduction

Github può hostare dei repository (o repo) che conservano il codice sorgente e possono essere pubblici o privati

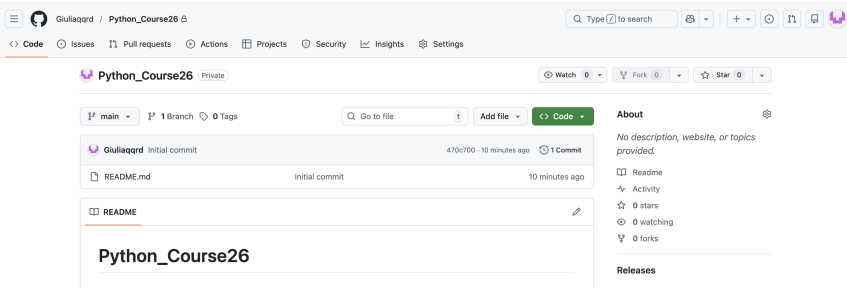


Figure: Git repo example

# Flusso di lavoro di Git



Il flusso di lavoro di **Git** si articola principalmente in tre aree:

1. **Working Directory**
   Area in cui si modificano i file del progetto.
2. **Staging Area**
   Area di preparazione dei file tramite il comando `git add`.
3. **Repository Git**
   Salvataggio definitivo delle modifiche nello storico del progetto tramite `git commit`.

I file passano dallo stato **modificato** allo stato **tracciato**, per poi essere consolidati nello storico del progetto.

# Table of Contents

2 Practice Basics


- Introduction
- Practice Basics
- Python Basics

# Usare il terminale

2 Practice Basics

Ogni sistema operativo ha dei comandi predefiniti da usare da terminale per svolgere diverse operazioni:

- creare / eliminare / spostarsi tra le directories
- stampare il contenuto delle directories
- creare, copiare, spostare o eliminare file
- visualizzare il percorso corrente
- eseguire programmi e script

# Percorsi relativi e assoluti



Un **percorso** indica la posizione di un file o di una directory nel file system.

**Percorso assoluto**

- parte dalla **radice** del file system
- descrive il percorso completo
- non dipende dalla directory corrente

Esempio: `/Users/nomeutente/Desktop/progetto/script.py`

**Percorso relativo**

- parte dalla **directory corrente**
- è più corto e dipende da dove ci troviamo nel file system

Esempio: `progetto/script.py`

# Comandi base del terminale

## 2 Practice Basics

| Azione | Windows (CMD / PowerShell) | Mac / Linux (Terminale) |
|---|---|---|
| Stampare la cartella corrente | `cd` oppure `pwd` | `pwd` |
| Mostrare file e cartelle | `dir` | `ls` |
| Entrare in una sottocartella | `cd nome_cartella` | `cd nome_cartella` |
| Tornare alla cartella precedente | `cd ..` | `cd ..` |
| Tornare alla cartella home | `cd %USERPROFILE%` | `cd` |
| Pulire il terminale | `cls` | `clear` |

# Esercizio: Navigare nelle cartelle



**Struttura del progetto**

Creare le cartelle usando `mkdir nome_cartella`

```
progetto/
  data/
    input/
    output/
  scripts/
```

**Obiettivo**

Partendo dalla cartella `progetto`:

1. Verificare in quale cartella ci si trova
2. Entrare nella cartella `data`
3. Visualizzare il contenuto della cartella
4. Entrare nella cartella `input`
5. Tornare alla cartella `data`
6. Tornare alla cartella `progetto`

**Suggerimento**

Usare i comandi `pwd`, `cd`, `ls` (o `dir` su Windows).

# Cosa ci serve per impostare l'ambiente di lavoro?



- Procedere con installazione di:
  - Package manager
  - Git
  - Python
- Creare account su github
- Scegliere un IDE (VSCode, Pycharm...)

# Table of Contents

3 Python Basics


- Introduction
- Practice Basics
- Python Basics

# Variabili



- Una variabile contiene un valore in un dato istante di tempo (ad un certo punto del codice)
- Il valore all'interno di una variabile può essere modificato nel tempo (per variabile, si intende che il valore può cambiare, quindi potremmo successivamente cambiare il valore della variabile `eta_mario`, ad esempio, da 19 a 20)
- Le variabili hanno un'**operazione di assegnamento**, ovvero bisogna indicare il nome della variabile, l'uguale e il valore che si vuole attribuire

# Variabili



- Una variabile può essere modificata
- Ad una variabile può essere associato un valore di un'altra variabile
- Una variabile può contenere un valore indefinito, detto nullo (`None`)
- Quando una variabile viene utilizzata per la prima volta nel programma e le viene assegnato un valore, si parla di **dichiarazione della variabile**.

# Operatori di Confronto



Esistono altre tipologie di operatori, tra cui gli **operatori di confronto**. Vengono utilizzati per confrontare due valori. Restituiscono il valore di verità, detto booleano (che può essere `True` o `False`).

- `==` : uguale
- `!=` : diverso
- `<` : minore
- `>` : maggiore
- `<=` : minore o uguale
- `>=` : maggiore o uguale

# Operatori di Confronto



Esempio:

```
a = 5
b = 10
print(a == b) # False
print(a != b) # True
print(a < b) # True
print(a > b) # False
print(a <= b) # True
print(a >= b) # False
```

# Funzioni



In programmazione, una `funzione` è una sequenza di istruzioni, identificata con un nome. Quando definisci una funzione, specifichi il nome e la sequenza di istruzioni. In seguito, puoi "chiamare" la funzione utilizzando il suo nome.

Alcune delle funzioni più comuni presenti in Python (built-in) sono:

- `int(...)  # converte il contenuto della funzione in intero`
- `str(...)  # converte il contenuto della funzione in stringa`
- `print(...)  # stampa il contenuto della funzione nel terminale`
- `input(...)  # richiesta in input`
- `type(...)  # restituisce il tipo dell'input fornito`
- ...

# Chiamare una funzione



Consideriamo questo esempio:

1. `a = 5`
2. `print(type(a))`

Risultato: `<class 'int'>`

- Il nome della funzione è `type`.
- L'espressione tra parentesi (ovvero, `a`) è chiamata `argomento` della funzione.
- Il risultato, per questa funzione, è il tipo dell'argomento, quindi `<class 'int'>`

# Funzioni matematiche



Python ha un `modulo` chiamato math che fornisce la maggior parte delle funzioni matematiche più comuni. Un modulo è un file che contiene una raccolta di funzioni che è possibile importare nel proprio programma.

Prima di poter utilizzare le funzioni di un modulo, dobbiamo importarlo con un'istruzione di importazione:

- `import math`

# Funzioni matematiche



Per accedere a una delle funzioni di `math`, devi specificare il nome del modulo e il nome della funzione, separati da un punto (noto anche come *dot notation*, notazione a punti). Ad esempio:

1. `import math`
2. `a = 9`
3. `radice_quadrata = math.sqrt(a)`

# Creare nuove funzioni



Finora abbiamo usato solo le funzioni già fornite da Python (*built-in*), ma è anche possibile crearne di nuove. Per definire una funzione bisogna indicare il nome della nuova funzione e la sequenza di istruzioni che verranno eseguite quando la funzione viene chiamata.

```
def saluta():
  print("Ciao!")
```

`def` è una parola chiave che indica che questa è una definizione di funzione. Il nome della funzione è `saluta`.

# Creare nuove funzioni



```
def saluta():
  print("Ciao!")
```

Alcune osservazioni:

- Le parentesi vuote dopo il nome indicano che questa funzione non prende argomenti.
- La prima riga della definizione della funzione si chiama intestazione (header); il resto si chiama corpo (body).
- L'intestazione deve terminare con due punti : e il corpo deve essere indentato.
- Il corpo può contenere un numero qualsiasi di istruzioni.
- Le funzioni create manualmente vengono richiamate all'interno del codice, come fatto con le funzioni built-in
- Le funzioni possono essere richiamate dentro altre funzioni

# Parametri e argomenti



Alcune delle funzioni che abbiamo visto richiedono argomenti. Ad esempio, quando chiami `math.sqrt` devi passare un numero come argomento. Alcune funzioni accettano più di un argomento: ad esempio, `math.pow`, che calcola un numero elevato a una potenza, ne richiede due, la base e l'esponente.

Ecco la definizione di una funzione che accetta un argomento:

```
def saluta(nome):
  print("Ciao",nome,"!")
```

# Variabili Locali



Quando crei una variabile all'interno di una funzione, essa è `locale`, il che significa che esiste solo all'interno della funzione. Ad esempio:

```
def saluta(first_name, last_name):
    full_name = first_name + last_name
    print("Ciao",full_name,"!")
```

Quando la funzione `saluta` termina, la variabile `full_name` viene distrutta. Se proviamo a stamparla all'interno del programma principale, otteniamo un'eccezione.

Attenzione, Anche i parametri sono locali.

# Condizione



Una **condizione** è un'espressione logica che può assumere solo due valori:

- **Vero** (true)
- **Falso** (false)

Esempio: data una variabile `eta`, possiamo verificare se una persona è maggiorenne con la seguente condizione:

```
eta >= 18
```

Questa condizione sarà:

- **vera** se `eta` ha un valore uguale a 18 o più
- **falsa** se `eta` ha un valore minore di 18

# Operatori logici



Per scrivere condizioni complesse nel costrutto **IF-ELSE**, possiamo combinare più condizioni usando gli **operatori logici**.

Le principali operazioni logiche sono:

- **AND** (congiunzione): è vera se entrambe le condizioni sono vere
- **OR** (disgiunzione): è vera se almeno una delle condizioni è vera
- **NOT** (negazione): inverte il valore di verità di una condizione

Le operazioni logiche si applicano a condizioni booleane (cioè condizioni che possono essere vere o false).

# Esempi di operatori logici



**Esempio AND** (congiunzione)
In italia, una persona può votare se ha almeno 18 anni **e** è cittadino italiano:

```
eta >= 18 AND cittadino_italiano == true
```

**Esempio OR** (disgiunzione)
Un cliente ha diritto allo sconto se è studente **o** pensionato:

```
studente == true OR pensionato == true
```

Verifichiamo se un pacco **non** è stato consegnato:

```
NOT consegnato
```

# Condizioni a catena



Quando ci sono più di due possibilità, si può usare una struttura con più rami, detta **condizione a catena**.

**Esempio**:

```
if x < y:
  print('x è minore di y')
elif x > y:
  print('x è maggiore di y')
else:
  print('x e y sono uguali')
```

# Condizioni annidate



È possibile inserire un `if` dentro un altro `if`, creando una struttura **annidata**.

**Esempio**:

```
if x == y:
  print('x e y sono uguali')
else:
  if x < y:
    print('x è minore di y')
  else:
    print('x è maggiore di y')
```

Le condizioni annidate sono utili, ma riducono la leggibilità.

# Ciclo While



**Definizione**

Il ciclo `while` esegue ripetutamente un blocco di codice finché una certa condizione rimane vera.

```
while condizione:
    istruzioni
```

**Esempio**

Stampare i numeri da 1 a 5:

```
i = 1
while i <= 5:
    print(i)
    i = i + 1
```

**Output:** 1 2 3 4 5

# Ciclo For



Il **ciclo for** è un altro costrutto che permette di realizzare cicli. È simile al ciclo while, ma è costruito appositamente per essere utilizzato quando il numero di iterazioni da fare è noto.

Nota: while e for sono equivalenti. Solitamente si usa il for quando il numero di cicli è noto e il while quando il numero di cicli non è noto a priori.

# Ciclo For



La sintassi del ciclo for è la seguente:

```
for i in range(a,b):
    <blocco di codice>
```

Il blocco di codice viene ripetuto con la variabile i che varia da a (compreso) a b (escluso), incrementandosi in automatico di 1 ad ogni iterazione. Quindi, all'inizio del ciclo for `i` avrà valore `a` e, quando `i` avrà valore `b`, il ciclo terminerà.

# Incremento del contatore



Di default, la variabile contatore (ad esempio, `i`) viene incrementata di 1 ad ogni iterazione. E' possibile specificare un ulteriore parametro in `range`, per indicare di quanto incrementare il contatore

```
for i in range(0,10,2):
  print(i)
```

In questo caso, `i` verrà incrementata di 2 ad ogni iterazione.

# Strutture dati



**Definizione**

Le strutture dati permettono di memorizzare e organizzare più valori all'interno di una singola variabile.

1. **Lista (`list`)**: collezione ordinata e modificabile di elementi.
2. **Tupla (`tuple`)**: collezione ordinata ma non modificabile.
3. **Insieme (`set`)**: collezione non ordinata di elementi unici.
4. **Dizionario (`dict`)**: collezione di coppie *chiave-valore*.

**Esempi**

```
lista = [1, 2, 3]
tupla = (1, 2, 3)
insieme = {1, 2, 3}
dizionario = {"nome": "Anna", "eta": 25}
```

# Python for Data Science

*Thank you for listening!*

*Any questions?*